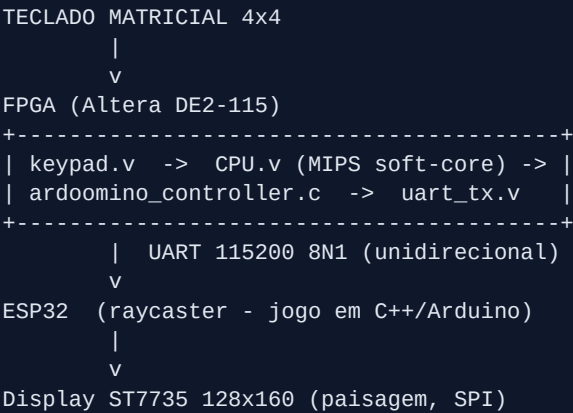


ArDOOMino

Guia de Apresentacao do Sistema

Roteiro para apresentar o fluxo completo: *tecla pressionada no teclado matricial* → *personagem reage no jogo*, passando por *FPGA*, *soft-core MIPS*, *UART* e *ESP32*.

1. Visao Geral da Arquitetura



Tres repositorios envolvidos:

Repositorio	Papel
C:\Projetos\Quartus	Verilog: scan do teclado, CPU MIPS customizada, transmissor UART
C:\Projetos\Compilador\156711	Compilador C-minus proprio + firmware (ardoomino_controller.c)
C:\Projetos\ArDOOMino\raycaster	Jogo em C++ (PlatformIO/Arduino) rodando no ESP32

2. Os 6 Estagios do Pipeline

Estagio 1 — Scan fisico do teclado (Verilog, FPGA)

Arquivo: Quartus/keypad.v

- Teclado matricial **4x4** (16 teclas: 1 2 3 A / 4 5 6 B / 7 8 9 C / * 0 # D).
- Um **divisor de clock** gera uma varredura mais lenta que os 50 MHz da placa.
- A cada ciclo de varredura, uma **coluna por vez** e ativada (round-robin) e as 4 linhas sao lidas.
- Decodificacao combinacional gera um codigo **one-hot de 16 bits** (1 bit unico por tecla).
- **Debounce**: a tecla fica "latched" ate **3 ciclos de varredura consecutivos** sem deteccao — evita ruido de contato.

Layout fisico x valor one-hot:

```
1=0x0001  2=0x0002  3=0x0004  A=0x0008
4=0x0010  5=0x0020  6=0x0040  B=0x0080
7=0x0100  8=0x0200  9=0x0400  C=0x0800
*=0x1000  0=0x2000  #=0x4000  D=0x8000
```

Estagio 2 — Leitura pela CPU soft-core (MIPS customizado)

Arquivo: Quartus/CPU.v

- Processador MIPS customizado, escrito em Verilog, sintetizado dentro da propria FPGA.
- Instrucao de maquina **READ_KEYPAD** (opcode 0x30) conecta o barramento de dados diretamente a saida segment do modulo de scan.
- Nao ha processamento intermediario: a CPU le o valor one-hot bruto.

Estagio 3 — Firmware decide qual caractere enviar

Arquivo: Compilador/156711/ardoomino_controller.c

- Programa em **C-minus** (subconjunto de C), compilado por um **compilador caseiro** (testCompiler.exe + testAssembler.exe) para o binario que vira a ROM da CPU.
- Loop infinito: le read_keypad(), compara com a ultima tecla (last); so age se mudou (edge-triggered).
- Ao soltar uma tecla, envia espaco (32) antes do proximo caractere.

```
key = read_keypad();
if (key != last) {
    if (last != 0) serial_write(32);          // tecla solta -> espaco
    if (key == 2)    serial_write(87); // '2' -> 'W' (frente)
    else if (key == 32) serial_write(83); // '5' -> 'S' (tras)
    else if (key == 16) serial_write(65); // '4' -> 'A' (esquerda)
    else if (key == 64) serial_write(68); // '6' -> 'D' (direita)
    else if (key == 1)  serial_write(81); // '1' -> 'Q' (girar esq.)
    else if (key == 4)  serial_write(69); // '3' -> 'E' (girar dir.)
    else if (key == 16384) serial_write(70); // '#' -> 'F' (atirar)
    else if (key == 4096) serial_write(82); // '*' -> 'R' (acao)
    last = key;
}
```

serial_write() e uma funcao *builtin* do compilador: traduzida diretamente para a instrucao de maquina UART_WRITE_CHAR.

Estagio 4 — Transmissao UART por hardware (FPGA → ESP32)

Arquivo: Quartus/uart_tx.v

- UART_WRITE_CHAR dispara um pulso de 1 ciclo (uart_tx_strobe).
- Modulo serializa o byte em **115200 baud, 8N1** (8 bits de dados, sem paridade, 1 stop bit).
- BIT_CLK_MAX = 50.000.000 / 115.200 ≈ 434 ciclos de clock por bit transmitido.
- Fica busy durante a transmissao; novas escritas aguardam.

Fiacao fisica:

FPGA (DE2-115)	ESP32
GPIO[35] / PIN_AG26 (saida uart_tx)	GPIO 32 (RX do Serial2)
GND	GND

Comunicacao **unidirecional** — so a FPGA transmite, o ESP32 nunca responde.

Estagio 5 — ESP32 recebe via SerialFPGA (UART2 de hardware)

Arquivo: raycaster/src/main.cpp

```
static HardwareSerial SerialFPGA(2); // UART2 do ESP32 (built-in, sem lib externa)

void setup() {
    SerialFPGA.begin(115200, SERIAL_8N1, 32, 33); // RX=32, TX=33 (nao usado)
}
```

- HardwareSerial vem embutida no **core Arduino do ESP32** (framework = arduino, pacote espressif32 no platformio.ini) — nao e biblioteca de terceiros.
- O ESP32 tem 3 UARTs fisicos: UART0 (=Serial, USB), UART1 (nao usado), UART2 (=Serial2).
- Gracias a **GPIO Matrix** programavel do ESP32, o UART2 pode ser roteado para quaisquer pinos (aqui, 32/33) em vez de pinos fixos.
- O periferico de hardware recebe os bits e armazena em FIFO; o driver copia para um buffer circular em RAM, exposto por available() / read().

```
InputData receiveData() {
    static InputData data = {0, 0, 0, true};
    data.flags &= ~((1<<7)|(1<<6)); // limpa shoot/acao a cada frame
    while (SerialFPGA.available()) { // drena todo o buffer
        char c = SerialFPGA.read();
        applyInputChar(c, data);
    }
    return data;
}
```

applyInputChar() converte cada caractere em campos da struct InputData:

Char	Efeito
W / S / A / D	seta bit de direcao em flags, zera giro
Q	x = +KEYPAD_TURN_RATE (gira esquerda)
E	x = -KEYPAD_TURN_RATE (gira direita)
F	bit de tiro em flags
R	bit de acao em flags

Char	Efeito
espaco	zera tudo (tecla solta)

InputData e **persistente** — o personagem continua andando/girando ate chegar um espaco (nao precisa reenviar a tecla a cada frame).

Estagio 6 — O jogo processa o input

Arquivo: raycaster/src/game/Player.hpp

A cada frame, `Player::update(input, dt)`:

- **Rotacao:** `_yaw -= input.x * dt * SENSITIVITY * DEG_TO_RAD` — campo x (de Q/E) gira a camera continuamente.
- **Direcao:** `input.getDirection()` resolve os bits de flags em um enum `Direction` (incluindo diagonais, ex. `W+A = UP_LEFT`).
- **Movimento:** direcao rotacionada relativa ao vetor de visao do personagem, aplicada com colisao contra o mapa.
- **Tiro:** `input.leftClick()` (bit do F) — respeitando cooldown (`SHOT_TIME`), aplica dano e alerta inimigos proximos.
- **Acao/porta:** `input.rightClick()` (bit do R) — abre porta se o jogador tem a chave certa.

O resultado e renderizado pelo raycaster e enviado via SPI ao display **ST7735 128x160** em paisagem (`setRotation(1)`).

3. Mapa de Teclas (layout atual)

Tecla fisica	Caractere UART	Acao no jogo
2	W	Mover para frente
5	S	Mover para tras
4	A	Strafe esquerda
6	D	Strafe direita
1	Q	Girar esquerda
3	E	Girar direita
#	F	Atirar
*	R	Acao (abrir porta)
soltar	espaco	Parar movimento/giro

4. Pontos-chave para a Apresentacao

- **Tres camadas tecnologicas distintas trabalhando juntas:** hardware digital puro (Verilog/FPGA), software embarcado em C-minus (compilador caseiro), e firmware em C++ Arduino-style (ESP32).
 - **Debounce e deteccao de borda em dois niveis:** primeiro em hardware (keypad.v, 3 ciclos de scan), depois em software (ardoomino_controller.c, comparacao `key != last`).
 - **Protocolo simples e robusto:** UART 115200 8N1 unidirecional, apenas 1 byte por evento de tecla (mais o espaco de "soltar"), sem handshake.
 - **Sem bibliotecas externas no ESP32 para a recepcao** — tudo via `HardwareSerial`, parte do core padrao arduino-esp32.
 - **Input persistente:** o jogo nao precisa de polling continuo do teclado — o estado fica retido ate o proximo evento.
 - **One-hot encoding** do teclado simplifica tanto o hardware de scan quanto a decodificacao em software (comparacoes diretas de valor inteiro).
-

5. Roteiro Sugerido de Demonstracao

1. Mostrar o teclado fisico e o LED de debug (LEDG[8]) acendendo a cada tecla pressionada.
2. Mostrar no monitor serial (tools/uart_sniffer) os bytes ASCII chegando conforme as teclas sao pressionadas/soltas.
3. Trocar para a ROM do jogo e mostrar o personagem respondendo no display.
4. (Opcional) Abrir o keypad.v e apontar a logica de debounce; abrir ardoomino_controller.c e mostrar o mapeamento tecla→char; abrir Player.hpp e mostrar como o char vira movimento.

6. Perguntas Frequentes Esperadas

“Por que não usar Arduino padrão para ler o teclado?”

O scan é feito direto em hardware (FPGA) para já integrar com a CPU MIPS didática do curso — não há microcontrolador adicional entre o teclado e o soft-core.

“Por que UART e não I2C/SPI para o link FPGA↔ESP32?”

UART é mais simples de implementar em Verilog do zero (sem necessidade de lógica de barramento/endereçamento) e suficiente para a taxa de eventos de um teclado.

“O ESP32 pode travar esperando dados?”

Não — `SerialFPGA.available()` é não-bloqueante; o loop do jogo roda independente de haver dados ou não.

“Como funciona o debounce duplo (hardware + software)?”

O hardware (`keypad.v`) filtra ruído elétrico de contato (glitches durante o scan). O software (`ardoomino_controller.c`) garante que o mesmo evento não seja reenviado repetidamente enquanto a tecla continua pressionada — só dispara em mudança de estado.